



StudyHub Production Deployment Runbook

PRIMARY DOMAIN

https://studyhubonline.store

AWS REGION & ELASTIC IP

ap-south-1 (Mumbai) • 15.206.223.108

DEPLOYMENT STATUS

LIVE & SSL SECURED

DATABASE

AWS RDS PostgreSQL 18.3 (studyhub-prod-db)

1. Production Architecture

```
[ Internet User / Browser ] | HTTPS (Port 443) ▼ [ AWS Elastic IP: 15.206.223.108 ] | [
Nginx Reverse Proxy + Let's Encrypt SSL ] |— studyhubonline.store / www —> 127.0.0.1:3000
(studyhub-web Next.js) |— admin.studyhubonline.store—> 127.0.0.1:3001 (studyhub-admin
Next.js) |— api.studyhubonline.store —> 127.0.0.1:5000 (studyhub-api Express) | [ Internal
Docker Network: studyhub-network ] |— studyhub-redis (Port 6379) |— studyhub-worker
(Background Queues / BullMQ) | VPC Security Group Inbound Rule (Port 5432) ▼ [ AWS RDS
PostgreSQL: studyhub-prod-db:5432 ]
```

2. Cloud & Infrastructure Inventory

Resource	Identifier / Value	Purpose & Notes
AWS VPC	vpc-080ee16ec1b11057e	CIDR: 10.0.0.0/16
EC2 Server	i-0cccd30b73fe597ae7	Ubuntu 24.04 LTS (t3.medium , 30GB gp3)
Elastic IP	15.206.223.108	Static Public IPv4 attached to EC2
EC2 Security Group	sg-006dabca578c89482	Inbound: Port 22 (SSH), 80 (HTTP), 443 (HTTPS)
RDS PostgreSQL	studyhub-prod-db	Endpoint: studyhub-prod-db.cngoksag0z0e.ap-south-1.rds.amazonaws.com
RDS Security Group	sg-0ef77fb4a0998ff9a	Inbound: Port 5432 open to VPC CIDR 10.0.0.0/16
Swap Space	/swapfile (4GB)	Prevents OOM during Next.js production builds

3. DNS Configuration (GoDaddy)

Type	Host	Target Value	TTL	Status
A	@	15.206.223.108	1/2 Hour	ACTIVE
A	www	15.206.223.108	1/2 Hour	ACTIVE
A	admin	15.206.223.108	1/2 Hour	ACTIVE
A	api	15.206.223.108	1/2 Hour	ACTIVE

4. Issues Encountered & Solutions Applied

⚠ Issue 1: SSH Port 22 Connection Timed Out

Cause: EC2 was launched in subnet `studyhub-private-2b` attached to a private route table routed only to a NAT Gateway.

Solution: Associated `studyhub-private-2b` with `studyhub-public-rt` (`rtb-032c88d4d3472f501`), routing directly to the Internet Gateway.

⚠ Issue 2: SSH Connection Reset & Out Of Memory (OOM) During Builds

Cause: Compiling Next.js 16 across multiple containers simultaneously consumed 100% CPU and exhausted physical RAM.

Solution: Created a 4GB Swap file (`/swapfile`), added `-o ServerAliveInterval=60` to SSH, and built containers sequentially.

⚠ Issue 3: Next.js Standalone Build in Monorepo

Cause: Next.js does not generate standalone output folders without explicit configuration.

Solution: Configured `output: 'standalone'` in `apps/web/next.config.ts` and `apps/admin/next.config.ts`.

⚠ Issue 4: Admin Docker Build Missing public/ Directory

Cause: `apps/admin` lacked a public folder, breaking the Dockerfile `COPY` step.

Solution: Added `RUN mkdir -p apps/admin/public` and created `apps/admin/public/.gitkeep`.

⚠ Issue 5: Prisma 7 Build-Time Generate Failure

Cause: Prisma 7 requires a `DATABASE_URL` environment variable even during client generation.

Solution: Passed a dummy build-time variable:

```
DATABASE_URL="postgresql://build:build@localhost:5432/build" npm run db:generate .
```

⚠ Issue 6: Node 24 ESM Module Resolution (API & Worker)

Cause: Node 24 ESM requires explicit `.js` extensions on compiled relative imports and ignores TypeScript path aliases.

Solution: Configured `Dockerfile.api` and `Dockerfile.worker` to execute with `npx tsx --tsconfig ...`.

⚠ Issue 7: Missing package.json in API Runtime Container

Cause: Runtime stage omitted root `package.json`, causing workspace npm commands to fail with `ENOENT`.

Solution: Added `COPY --from=build /app/package.json /app/package-lock.json ./` to both Dockerfiles.

⚠ Issue 8: RDS PostgreSQL Port 5432 Connection Timed Out

Cause: The RDS Security Group only allowed incoming traffic from itself, blocking the EC2 server (`10.0.87.55`).

Solution: Added an Inbound Rule to `sg-0ef77fb4a0998ff9a` for PostgreSQL port 5432 from `10.0.0.0/16` (VPC CIDR).

5. Server Maintenance Runbook

Connecting to Server

```
ssh -o ServerAliveInterval=60 -i "C:\Users\Sanjay\Downloads\studyhub-key.pem" ubuntu@15.206
```

Container Management

```
cd ~/studyhub
# Check status of all services
docker compose -f docker-compose.prod.yml ps

# View live container logs
docker logs -f studyhub-api
docker logs -f studyhub-web
docker logs -f studyhub-admin
docker logs -f studyhub-worker
```

Applying Database Migrations

```
docker compose -f docker-compose.prod.yml exec -w /app/packages/database api npx prisma mig
```

Deploying Code Updates

```
cd ~/studyhub
git pull origin main
docker compose -f docker-compose.prod.yml up -d --build <service-name>
```

6. Live Production URLs

- ✓ **Main Web Application:** <https://studyhubonline.store>
- ✓ **Admin Management Portal:** <https://admin.studyhubonline.store>
- ✓ **Backend API Health:** <https://api.studyhubonline.store/api/v1/health/ready>